Programming Problems

Problem Set

Problem 1 Maximum Sub-array of sum exactly K

Source: Extension of the question taught in class

You are given an array $A[1..n]$ of nonnegative integers and a positive integer K. Find the maximum length of a contiguous subarray whose elements sum to exactly K. Formally, determine

$$\max\left\{ r-\ell+1 :\ 1 \le \ell \le r \le n, \quad \sum_{j=\ell}^{r} A[j] = K \right\}.$$

Return 0 if no such subarray exists. Design a bottom-up dynamic programming algorithm that solves this problem.

We looked at the definition of the DP that works for this problem ruling out the base definitions.

Problem 2 The Sports Festival

Source: https://codeforces.com/contest/1509/problem/C

The student council is preparing for the relay race at the sports festival. The council consists of n members. The speed of member i is s_i.

The discrepancy d_i of the i-th stage is the difference between the maximum and minimum running speed among the first i members who ran. Formally, if a_i denotes the speed of the i-th member who participated in the race, then

$$d_i = \max(a_1, \ldots, a_i) - \min(a_1, \ldots, a_i).$$

You may change the order in which the members run. Find the minimum possible value of

$$d_1 + d_2 + \cdots + d_n.$$

We looked at an observation which helped us design a DP based algorithm.

Problem 3 Simple String

Source: https://codeforces.com/problemset/problem/665/C
Given a string, call it *simple* if no two adjacent characters are equal. In one operation, you may change any character of the string to any other character. Find the minimum number of operations required to make the given string simple.

We looked at the following algorithms:

1. Greedy: in $O(n)$.
2. DC: using more solutions at each stage with the combine step performed in $O(1)$ time, making the algorithm run in $O(n)$ time.
3. DP: in $O(n)$ time.

Problem 4 Tower Placements by Number of Active Nodes

Source: Harness OA, 2026

You are given a tree with N nodes. You may place towers on any subset of its nodes. A tower activates its own node and all directly connected neighbors. A node is called *active* if it is covered by at least one tower.

For every k from 0 to N, determine the number of distinct tower-placement configurations that activate exactly k nodes.

We have not discussed this.

Problem 5 Staircase problem

Source: Question 8 of Mid-sem.

Consider a staircase consisting of n steps. Stepping on step i incurs a cost c_i. Some of the steps are unavailable and cannot be used. From any step, you may move forward by 1, 2, or 3 steps.

Design a dynamic programming algorithm to find a minimum-cost way of reaching the n-th step, or determine that it is impossible to reach the n-th step. Clearly specify the DP state, recurrence relation, and initialization. Also explain how an optimal sequence of steps can be retrieved. Analyze the time and space complexity of your algorithm.

We looked at a D&C algorithm for this problem that requires $O(1)$ in the combine steps; this storing more solutions

Problem 6 Charging points

Source: Question 6 of Mid-sem.

A vehicle has to travel along a straight road from position 0 to position D. There are n charging stations located at positions $x_1 < x_2 < \cdots < x_n$. After charging, the vehicle can travel at most L units before it needs to charge again. Design an efficient greedy algorithm that minimizes the number of charging stops required to reach the destination, or reports that the journey is impossible. State the greedy-safeness property for your algorithm and prove it. Also state the optimal substructure property.

We looked at a greedy algorithm for this problem. Also discussed the greedy safeness, its correctness and the statement of optimal substructure property.